

Fiche de révision — Terraform & Infrastructure as Code (TP2)

Support de révision du cours magistral « *Infrastructure as Code avec Terraform sur Azure* » — Bloc 4, Mastère Dev Manager Full Stack. Cas fil rouge : **ShopEasy**.

1. Pourquoi l'Infrastructure as Code (IaC) ?

Le problème : créer des ressources à la main dans le portail Azure est rapide pour découvrir, mais ingérable dès qu'on gère plusieurs environnements (dev / test / préprod / prod). Les manipulations manuelles ne se rejouent jamais à l'identique : une règle réseau oubliée, un tag manquant... et deux environnements censés être identiques divergent.

L'IaC : on décrit l'infrastructure dans des **fichiers texte** (réseaux, VM, NSG, stockage...). Ces fichiers deviennent la **source de vérité**, versionnés dans Git, relus en revue de code, testés, appliqués et détruits de façon répétable. *L'infrastructure devient un actif logiciel.*

Bénéfices entreprise : reproductibilité, traçabilité (Git), réduction des erreurs, standardisation, vitesse de livraison, auditabilité, optimisation des coûts.

Cas entreprise. Une banque doit fournir une préprod **identique** à la prod pour valider une mise à jour réglementaire. En manuel, le moindre écart fausse les tests. En IaC, le même code génère un clone fidèle en 15 min, et la conformité est auditable via l'historique Git.

2. Terraform : déclaratif et multi-cloud

Terraform (HashiCorp) décrit l'infra en **HCL**. Il est **déclaratif** : on décrit *l'état final voulu*, et Terraform calcule lui-même les actions pour l'atteindre — au lieu de lister les étapes une à une (impératif).

Approche	Principe	Exemple
Impérative (script)	Décrit les étapes	« crée un VNet, puis un subnet, puis attache un NSG »
Déclarative (Terraform)	Décrit l'état attendu	« il doit exister un VNet avec 2 subnets et un NSG associé »

Terraform est **multi-cloud** via les **providers** (plugins) : **azurerm** (Azure, celui du TP), mais aussi AWS, GCP, Kubernetes, GitHub... Providers Azure : **AzureRM** (standard, le TP), AzAPI (API récentes), AzureAD (identités Entra ID).

Cas entreprise. Une scale-up héberge son app sur Azure et son analytics sur GCP. Avec un seul outil et deux providers, l'équipe DevOps gère les deux clouds avec la même logique, au lieu d'apprendre deux outils natifs distincts.

3. Le workflow Terraform (le cœur du TP)

init → fmt → validate → plan → apply → output → destroy

Commande	Rôle	Analogie
init	Télécharge les providers, prépare le dossier	installer les dépendances
fmt	Met en forme le code	prettier
validate	Vérifie syntaxe et cohérence	compiler
plan	Prévisualise créations / modifs / destructions	git diff avant commit
apply	Applique réellement	déployer
output	Affiche les infos utiles (IP, noms...)	le reçu
destroy	Supprime tout proprement	nettoyer

Symboles du **plan** (à connaître absolument) :

Symbole	Signification
+	ressource à créer
-	ressource à détruire
~	ressource à modifier
-/+	à remplacer : détruire puis recréer (signal d'alerte)

Cas entreprise. Avant un déploiement en prod, un ingénieur lit le plan et voit **-/+** `azurerm_linux_virtual_machine` : destruction + recréation → interruption possible. Il bloque l'apply et planifie l'opération en heures creuses. **Le plan a évité un incident.**

4. Structure d'un projet & langage HCL

On découpe par responsabilité (`network.tf`, `compute.tf`, `storage.tf`, `variables.tf`, `outputs.tf`...) plutôt qu'un énorme `main.tf`. *Un projet doit être compris par quelqu'un qui ne l'a pas écrit.*

```
resource "azurerm_resource_group" "rg" { # type + nom logique (interne au code)
  name      = "rg-shopeasy-dev"
  location = "francecentral"
}
```

On **référence** une ressource depuis une autre (`azurerm_resource_group.rg.name`). Terraform en déduit l'ordre de création : le **graphe de dépendances** (dépendances implicites ; `depends_on` seulement si nécessaire).

Cas entreprise. Une équipe de 8 personnes maintient l'infra. Grâce au découpage clair, un nouvel arrivant comprend l'architecture en lisant le code, sans réunion. Lisibilité = onboarding rapide et moins d'erreurs.

5. Variables, locals, outputs (la règle d'or)

- **Variable** = ce qui **entre** (paramètre : région, taille de VM, environnement...) → évite le code en dur.
- **Local** = ce qui est **calculé** dans le projet (préfixe `shopeasy-dev`, tags communs...).
- **Output** = ce que le projet **expose** après déploiement (IP du Load Balancer, nom du RG...).

Cas entreprise. Le même code sert pour dev, test et prod : on change seulement `terraform.tfvars` (`environment = "prod"`, `vm_size = "Standard_D4s"`). Un seul code, trois environnements → zéro duplication, zéro divergence.

6. Le state Terraform (notion clé)

Le **state** (`terraform.tfstate`) fait le **lien entre le code et les ressources réelles** dans Azure. Sans lui, Terraform ne sait pas ce qu'il gère déjà.

Risques du state local : perte du fichier, conflits multi-utilisateurs, pas de verrouillage, **peut contenir des secrets**. → C'est pourquoi le `.gitignore` exclut `*.tfstate` : **on ne met jamais le state dans Git**.

En entreprise → **state distant** : Storage Account Azure dédié (backend `azurerm`), avec verrouillage et droits RBAC.

```
terraform {
  backend "azurerm" {
    resource_group_name = "rg-terraform-state"
    storage_account_name = "sttfstateprod001"
    container_name      = "tfstate"
    key                 = "shopeasy/dev/terraform.tfstate"
  }
}
```

Cas entreprise. Deux ingénieurs lancent `apply` simultanément. Avec un state distant **verrouillé**, le second est mis en attente → pas de corruption. Avec un state local, ils écrasaient mutuellement leurs changements.

7. Authentification & secrets

- **En formation** : `az login` suffit (Terraform réutilise la session CLI).
- **En entreprise** : Terraform tourne dans un **pipeline CI/CD** avec un **service principal** ou une **identité managée**, aux droits **limités au périmètre** (jamais Owner sur tout l'abonnement).

- **Secrets** : jamais dans les `.tf` ni les `.tfvars` versionnés → **Azure Key Vault**, variables d'environnement, variables sécurisées CI/CD, rotation régulière.

Cas entreprise. Un dev pousse par erreur un `.tfvars` contenant un mot de passe sur GitHub. Key Vault + scan de secrets dans la CI auraient bloqué l'incident — classique et coûteux.

8. Drift — quand le réel diverge du code

Le **drift** = écart entre le code et la réalité Azure. Typiquement : quelqu'un modifie **à la main** un NSG dans le portail. Au prochain `terraform plan`, Terraform **détecte l'écart** et propose de revenir à l'état déclaré.

Pourquoi c'est dangereux : l'infra devient imprévisible, la sécurité s'affaiblit, on perd confiance dans le code comme source de vérité.

Réduire le drift : limiter les modifs manuelles, `terraform plan` régulier, pipelines, Azure Policy, tout changement par pull request.

Cas entreprise. Un admin ouvre SSH en urgence un vendredi soir et oublie. Trois semaines plus tard, un audit trouve le port ouvert. Un `plan` régulier l'aurait repéré dès le lundi. **Règle : tout passe par le code, pas par le portail.**

9. Modules — réutiliser

Quand le projet grossit, on regroupe la logique réutilisable en **modules** (réseau, VM, stockage...), appelés avec des paramètres. Un module doit être assez générique pour être réutilisable, mais pas au point d'en devenir incompréhensible.

Cas entreprise. L'équipe plateforme publie un **module réseau standardisé** (VNet + subnets + NSG conformes à la politique sécurité). Les 12 équipes produites l'appellent en 5 lignes au lieu de réécrire (et mal configurer) le réseau. Standardisation = sécurité + gain de temps à l'échelle.

10. FinOps avec Terraform

Terraform aide à maîtriser les coûts car il rend les ressources **visibles, taguées et destructibles** — mais ne garantit pas l'optimisation sans gouvernance. Leviers : tailles de VM adaptées à l'environnement, tags de coût, **détruire les environnements temporaires**, éviter les IP publiques inutiles, estimer via Pricing Calculator.

Cas entreprise. Des environnements de test tournent H24 le week-end pour rien (milliers d'€/mois). Un tag `expiration` + un `terraform destroy` automatisé le vendredi soir font chuter la facture. L'objectif : le meilleur **rapport coût / valeur**, pas le coût minimum.

11. Terraform vs ARM / Bicep / scripts

Outil	Force	Limite	Cas d'usage
Terraform	Multi-cloud, modules, plan lisible	Gestion rigoureuse du state	Orga multi-cloud / IaC transverse
ARM Templates	Natif Azure, complet	JSON verbeux	Déploiements très proches de l'API Azure
Bicep	Natif Azure, lisible	Peu multi-cloud	Équipes 100 % Azure
Scripts CLI	Simples, ponctuels	Peu déclaratifs, peu reproductibles	Dépannage, automatisations ciblées

Les 5 réflexes à retenir pour le TP2

1. **Toujours lire le plan avant apply** (surtout les `-/+`).
2. **Le state ne va jamais dans Git** (et idéalement en backend distant en équipe).
3. **Pas de secret en clair** dans le code.
4. **Tout est tagué et nommé** selon une convention.
5. **Aucune modif à la main** dans le portail → tout passe par le code (sinon drift).

Glossaire express

Terme	Définition
IaC	Gestion d'infrastructure par fichiers versionnés
HCL	Langage de configuration de Terraform
Provider	Plugin pilotant une plateforme (ex. <code>azurerm</code>)
Resource	Élément géré (VNet, VM, Storage...)
Variable / Local / Output	Entre / calculé / exposé
State	Lien entre le code et les ressources réelles
Backend	Emplacement de stockage du state
Plan / Apply	Prévisualisation / application des changements
Drift	Écart entre code et infrastructure réelle
Module	Ensemble réutilisable de configurations

Note déploiement (abonnement Azure for Students). Le cours utilise `francecentral` et `Standard_B1s`, refusés par cet abonnement. Utiliser `location = "swedencentral"` et `size =`

"Standard_B2ts_v2" dans terraform.tfvars.